Recommendation System with Implicit Feedback

Thien, Nguyen Van Minh
22127398
Faculty of Information Technology
University of Science, VNU-HCM
nvmthien22@clc.fitus.edu.vn

Ky, Dang Van
22127227
Faculty of Information Technology
University of Science, VNU-HCM
dvky22@clc.fitus.edu.vn

***Abstract*—This report studies recommendation systems with implicit feedback under the binary Top-K setting, where observed user–item interactions are treated as positive signals and unobserved interactions are interpreted as missing feedback rather than true negatives. The report reviews the development of recommendation methods for implicit feedback and focuses on graph-based collaborative filtering, which is especially suitable for sparse user–item interaction data. Three representative state-of-the-art models—LightGCN, SGL, and SimGCL—are selected and analyzed because they form a clear methodological progression from simplified graph propagation to self-supervised contrastive learning and its later simplification. To support a fair comparison, the report adopts a unified offline evaluation protocol with shared preprocessing, data splitting, candidate-item construction, and metric computation across all models. The analysis emphasizes ranking-based evaluation, primarily Recall@K and NDCG@K, since these metrics reflect both retrieval effectiveness and ranking quality in Top-K recommendation. The goal of this report is to clarify the core ideas, innovations, and trade-offs of modern graph-based recommendation methods for implicit-feedback collaborative filtering.**

***Index Terms*—recommender systems, implicit feedback, graph-based recommendation survey, top-k recommendation survey**

I. Introduction

Recommendation systems play an important role in modern online platforms, including e-commerce, social media, video streaming, and news recommendation. By filtering large volumes of available content and prioritizing items that are likely to match user interests, they improve both user experience and platform engagement.

In recommendation research, user preference is commonly represented through **explicit feedback** or **implicit feedback**. Explicit feedback refers to signals that users provide deliberately and directly, such as ratings, likes/dislikes, or written reviews. These signals are usually clear and easy to interpret because they directly reflect user opinions. However, they are often sparse, since only a limited number of users actively provide such feedback. In contrast, implicit feedback is inferred indirectly from user behavior, such as clicks, views, purchases, likes, or watch time. Its main advantage is that it is abundant and naturally collected at scale in real-world systems. However, it is also noisier and more ambiguous, because an observed interaction does not always indicate genuine preference, while an unobserved interaction does not necessarily imply dislike.

This report focuses on recommendation systems with **implicit feedback** under the **binary interaction setting**. In this representation, an observed user-item interaction is encoded as 1, indicating that the user has interacted with the item, while an unobserved interaction is encoded as 0. Importantly, this binary formulation does not imply that 0 always represents a true negative preference; rather, it usually indicates missing feedback, since the user may simply have never been exposed to the item. Despite this limitation, binary interaction modeling is widely adopted because it simplifies the learning problem and is well suited to large-scale recommendation scenarios.

The task considered in this report is **Top-K recommendation**. Given a user, the system predicts preference scores for candidate items, ranks them, and returns the top K items that are most likely to be relevant to that user. Unlike rating prediction, Top-K recommendation emphasizes the quality of the ranked list, making it more consistent with the way recommendation systems are deployed in practice. In this setting, the goal is not to estimate an exact preference value, but to identify which items should appear near the top of the recommendation list.

Based on this problem setting, our study examines how different state-of-the-art recommendation models perform under a unified experimental framework. The scope of this work is limited to implicit feedback, binary interaction representation, Top-K recommendation, and ranking-based evaluation. In particular, we are interested in comparing representative recommendation architectures, understanding their strengths and limitations, and analyzing how dataset characteristics affect ranking performance under a fixed evaluation protocol.

II. Related Work

This section reviews the development of recommendation methods for implicit feedback, summarizes the main research directions, and positions the three models selected in this report.

A. Development Overview

Recommendation systems for implicit feedback have evolved through several major stages. Early work mainly focused on classical collaborative filtering methods, especially neighborhood-based models and matrix factorization. In this setting, user preference is inferred only from historical interaction behavior, without requiring explicit ratings. A foundational study by Hu, Koren, and Volinsky [1] showed that implicit-feedback recommendation differs fundamentally from explicit-feedback recommendation because it lacks reliable negative feedback, is

inherently noisy, and should be interpreted in terms of confidence rather than direct preference. This work also established the importance of designing models and evaluation protocols specifically for implicit-feedback data.

More broadly, the recommender-systems literature has gradually evolved from neighborhood methods to representation-learning approaches. According to the GNN in RS survey, early item-based neighborhood methods were attractive because of their simplicity, efficiency, and practical effectiveness, while later representation-learning methods encoded users and items as embeddings in a shared latent space. Matrix factorization became especially influential after the Netflix Prize era, and was later extended by neural recommendation methods that aimed to capture more complex and non-linear user–item relations. [2]

B. Main Research Branches

For the implicit-feedback setting considered in this report, the literature can be broadly grouped into several families. The first family is **neighborhood-based methods**, which estimate user preference from similar users or similar items. Their main advantage is interpretability and operational simplicity, but they are limited in modeling more complex collaborative patterns. The second family is **matrix factorization-based methods**, which learn latent user and item vectors and predict relevance through their interactions. These methods are more expressive and scalable than pure neighborhood models, but they still rely mainly on low-order collaborative structure. The third family is **neural recommendation methods**, which introduce non-linear representation learning and can model richer interactions, but often at the cost of higher complexity and weaker interpretability.

A particularly important branch for this report is **graph-based recommendation**. The GNN in RS survey emphasizes that much of the data in recommender systems naturally has graph structure: user–item interactions form a bipartite graph, user relations may form a social graph, and item attributes may be connected through a knowledge graph. This graph perspective provides a unified way to model collaborative signals and other structured information. More importantly, unlike traditional methods that only use user–item interactions as supervision, graph neural networks can explicitly exploit the topological structure of the interaction graph and propagate information across multi-hop neighborhoods. This makes graph-based recommendation especially suitable for collaborative filtering under sparse implicit feedback. [2]

C. Graph-based Collaborative Filtering

Within graph-based recommendation, one influential line of work focuses on **user–item collaborative filtering** over the interaction graph. Earlier graph-based collaborative filtering methods such as NGCF [3] attempted to adapt graph convolutional networks to recommendation by combining neighborhood aggregation with feature transformation and nonlinear activation. These models demonstrated that multi-hop message passing on the user–item graph can improve representation learning beyond conventional matrix factorization. However, later studies questioned whether all standard GCN components are really necessary in the collaborative-filtering setting.

LightGCN [4] represents a key simplification in this line of work. Instead of inheriting the full GCN design, it keeps only the essential neighborhood aggregation mechanism and removes feature transformation matrices and nonlinear activation functions. This simplification makes the model lighter, easier to optimize, and empirically very strong, which is why LightGCN has become a widely used backbone in graph collaborative filtering and a common reference point for later studies. In the broader evolution of graph-based recommendation, it can be seen as a representative strong baseline that captures the core benefits of graph propagation without unnecessary architectural complexity.

D. Self-supervised and Contrastive Learning for Recommendation

More recently, self-supervised learning and contrastive learning have become important directions in recommendation research, especially for sparse implicit-feedback data. The main motivation is that user–item interaction data are often highly sparse, so supervised recommendation loss alone may be insufficient to learn robust user and item representations. Contrastive learning addresses this issue by extracting auxiliary self-supervised signals directly from raw interaction data, without requiring additional labels. A common design is to create two different views of the original user–item graph and train the model to maximize representation consistency between these views.

SGL [5] is one of the representative methods in this direction. It extends graph collaborative filtering by combining a LightGCN-style encoder with graph augmentation and an InfoNCE-based contrastive objective. The motivation is to improve robustness under sparse, noisy, and long-tail interaction data. In this sense, SGL marks an important shift from purely supervised graph recommendation toward self-supervised graph learning for recommendation.

SimGCL [6] further revisits this line of work and questions whether graph augmentation is truly the essential reason behind the gains of contrastive learning. Its analysis shows that the contrastive objective itself plays the dominant role, while graph augmentation may contribute much less than previously assumed. Based on this insight, SimGCL discards complex graph perturbation and instead generates contrastive views by adding controlled random noise in the embedding space. The paper reports that this simpler design not only preserves the benefits of contrastive learning, but also improves efficiency and often achieves better recommendation performance than augmentation-based counterparts.

E. Positioning of the Selected Models

The three selected models are chosen because they best match the scope and purpose of this report. Since our study focuses on implicit-feedback collaborative filtering under the binary Top-K setting, *graph-based models are a natural choice because they directly exploit the user–item interaction graph, which is the main source of learning signal in this problem*. More specifically, the three models also represent

a clear methodological progression: LightGCN is selected as *a strong and widely adopted graph collaborative filtering baseline*, SGL is chosen because it *extends this backbone with self-supervised contrastive learning to better handle sparsity and noisy interactions*, and SimGCL is included as a more recent refinement that *simplifies graph augmentation while maintaining strong accuracy and better efficiency*. In addition, all three models are *well established in the literature*, have *public implementations*, and are *feasible to reproduce under our computational budget*, making them suitable for a fair and meaningful comparison.

III. Selected Models

A. *LightGCN*

a) *Core Idea:*

LightGCN is built on the intuition that the two most common components of Graph Convolutional Networks (GCNs) — feature transformation matrices and nonlinear activation functions — are actually redundant and harmful when applied to collaborative filtering with implicit feedback. Instead, the only meaningful operation is to let each user and item *aggregate* the embeddings of its neighbors on the user–item interaction graph. A user who interacted with many popular items will absorb those items' signals; an item interacted with by many active users will absorb those users' signals. Stacking multiple such propagation layers captures higher-order connectivity (friends-of-friends effects), and the final representation is a weighted combination of all layer outputs.

This design reflects an important property of collaborative filtering data. Unlike node-classification tasks, where graph neural networks operate on rich semantic node features, recommendation models usually start from trainable user and item ID embeddings. As a result, repeated feature transformation contributes little additional information, whereas neighborhood propagation is the component that truly captures collaborative signals from the interaction structure.

b) *Architecture:*

LightGCN operates on a bipartite user–item graph $\mathcal{G} = (\mathcal{U} \cup \mathcal{I}, \mathcal{E})$, where an edge $(u, i) \in \mathcal{E}$ exists if user u has interacted with item i.

Layer-wise propagation. At layer k, each node aggregates its first-order neighbors without any learnable weight matrix or activation:

$$\boldsymbol{e}_u^{(k)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|}\sqrt{|\mathcal{N}_i|}} \boldsymbol{e}_i^{(k-1)} \tag{1}$$

$$\boldsymbol{e}_i^{(k)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i|}\sqrt{|\mathcal{N}_u|}} \boldsymbol{e}_u^{(k-1)}. \tag{2}$$

Layer combination. The final embedding is the mean of all $K+1$ layer outputs, with layer-0 being the trainable ID embedding:

$$\boldsymbol{e}_u = \sum_{k=0}^{K} \alpha_k \boldsymbol{e}_u^{(k)}, \quad \boldsymbol{e}_i = \sum_{k=0}^{K} \alpha_k \boldsymbol{e}_i^{(k)}, \tag{3}$$

where $\alpha_k = \frac{1}{K+1}$ by default.

Prediction and training. The predicted relevance score is the inner product $\hat{y}_{ui} = \boldsymbol{e}_u^\top \boldsymbol{e}_i$. The model is trained with the Bayesian Personalised Ranking (BPR) loss over observed positive and sampled negative pairs. The only trainable parameters are the $|\mathcal{U}| + |\mathcal{I}|$ initial ID embeddings of dimension d.

c) *Innovation:*

Prior graph-based recommenders such as NGCF largely transferred the full GCN design from node-classification tasks into collaborative filtering. LightGCN's main innovation is to challenge this transfer directly and show that, in recommendation, feature transformation and nonlinear activation are not essential and may even hurt performance. Its contribution is therefore not merely architectural simplification for efficiency, but a task-specific reformulation of graph convolution for collaborative filtering. By keeping only normalized neighborhood aggregation and combining embeddings from multiple propagation depths, LightGCN captures high-order collaborative signals with fewer parameters, easier optimization, and better empirical performance.

d) *Reason for Selection:*

LightGCN is the de-facto backbone for graph-based collaborative filtering and serves as a direct baseline against more advanced methods. It is widely reproduced, achieving strong results on Gowalla (Recall@20 = 0.1830, NDCG@20 = 0.1554), Yelp2018 (Recall@20 = 0.0649, NDCG@20 = 0.0530), and Amazon-book (Recall@20 = 0.0411, NDCG@20 = 0.0315). Its simplicity makes it easy to implement, audit, and extend, and its public PyTorch implementation ensures full reproducibility within our computational budget. More importantly, LightGCN serves as the conceptual backbone of our model set: it establishes the simplified graph collaborative filtering framework upon which both SGL and SimGCL are later built and compared.

B. *Self-supervised Graph Learning (SGL)*

a) *Core Idea:*

SGL addresses a fundamental weakness of LightGCN: the model learns only from *observed* positive interactions, leaving the vast majority of the interaction space unexplored. Users with few interactions (long-tail users) receive poor representations because there is simply not enough supervision signal. SGL introduces a self-supervised auxiliary task on top of the main recommendation objective. By creating two randomly *augmented views* of the user–item graph and training the model to produce consistent representations of the same node across views, SGL derives additional supervision from the graph structure itself — without requiring any extra labels.

In this sense, SGL improves recommendation not simply by adding another objective, but by injecting additional supervisory signal when observed interactions alone are too sparse to shape reliable user and item embeddings. The contrastive task regularizes the encoder to preserve what remains stable across perturbed graph views, making the learned representations more robust to missing edges, noisy interactions, and long-tail sparsity.

b) *Architecture:*

SGL wraps LightGCN with a graph-augmentation and contrastive-learning module.

Graph augmentation. Three stochastic operators produce an augmented graph $\tilde{\mathcal{G}}$ from the original $\mathcal{G}$:

- **Node Dropout (ND)**: randomly mask a fraction ρ of users or items and all their edges.
- **Edge Dropout (ED)**: randomly remove a fraction ρ of edges.
- **Random Walk (RW)**: for each node, sample a fixed-length random walk to define its local neighbourhood.

Two independent augmented views $\tilde{\mathcal{G}}'$ and $\tilde{\mathcal{G}}''$ are generated at each training step.

Contrastive objective. LightGCN is applied to both views to obtain embeddings $\boldsymbol{z}_{u'}$ and $\boldsymbol{z}_{u''}$ for each user u (and analogously for items). The InfoNCE contrastive loss maximises agreement between the two views of the same node while repelling all other nodes:

$$\mathcal{L}_{\text{ssl}} = \sum_{u \in \mathcal{U}} -\log \frac{\exp\left(\text{sim}\, \frac{\boldsymbol{z}_{u'}, \boldsymbol{z}_{u''}}{\tau}\right)}{\sum_{v \in \mathcal{U}} \exp\left(\text{sim}\, \frac{\boldsymbol{z}_{u'}, \boldsymbol{z}_{v''}}{\tau}\right)}, \tag{4}$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity and τ is a temperature hyper-parameter. A symmetric term over items is added.

Joint training. The final objective is:

$$\mathcal{L} = \mathcal{L}_{\text{BPR}} + \lambda \mathcal{L}_{\text{ssl}}, \tag{5}$$

where λ balances the two tasks. The backbone weights are shared across views so contrastive training directly regularises the main recommendation embeddings.

c) *Innovation:*

SGL is one of the first works to systematically introduce self-supervised contrastive learning into graph-based collaborative filtering. Its innovation lies not only in combining graph augmentation with an InfoNCE objective, but in showing that recommendation can benefit from an auxiliary learning signal derived from the interaction graph itself. By enforcing consistency between multiple stochastic views of the same user or item, SGL encourages the encoder to learn representations that are more robust to sparsity, structural noise, and long-tail interaction patterns. In this sense, SGL marks an important methodological shift from purely supervised graph recommendation toward self-supervised graph learning for recommendation.

d) *Reason for Selection:*

SGL consistently outperforms LightGCN and all pre-2021 CF baselines on standard benchmarks, with reported gains of roughly 10–15% in Recall@20: Gowalla (0.2263 / 0.1917), Yelp2018 (0.0815 / 0.0659), Amazon-book (0.0712 / 0.0556). It represents the first successful integration of SSL into the GNN-based CF paradigm, making it a key model for understanding how auxiliary signals improve recommendation quality. Its public implementation and compatibility with LightGCN allow direct ablation experiments within the same framework. Methodologically, SGL is essential in our model set because it represents the transition from a strong supervised graph CF backbone to a self-supervised recommendation framework, making it possible to analyze what additional gains contrastive learning contributes beyond pure graph propagation.

C. *Simple Graph Contrastive Learning (SimGCL)*

a) *Core Idea:*

SimGCL revisits SGL and asks: *is graph augmentation actually the source of the performance gains, or is it something else?* Through theoretical analysis of the *uniformity–alignment* decomposition of the InfoNCE loss, the authors show that what truly matters is the *uniformity* pressure that contrastive learning exerts on the embedding space — not the specific graph perturbation strategy. Based on this finding, SimGCL replaces the expensive graph-augmentation module with the simplest possible operation: adding small uniform random noise directly to the node embeddings to generate contrastive views. The result is a model that is simultaneously simpler, faster, and more accurate than SGL.

Therefore, SimGCL should not be viewed merely as a lighter implementation of SGL. Its deeper contribution is to disentangle where the benefit of contrastive recommendation actually comes from, and to argue that the key improvement lies in how the contrastive objective reshapes the geometry of the embedding space rather than in handcrafted graph perturbations themselves.

b) *Architecture:*

SimGCL keeps the LightGCN encoder intact and changes only how contrastive views are produced.

Noise-based view generation. Given the final LightGCN embedding $\boldsymbol{e}_u$ of user u, two perturbed views are created by adding independent random noise vectors:

$$\boldsymbol{z}_{u'} = \boldsymbol{e}_u + \boldsymbol{\Delta}_{u'}, \quad \boldsymbol{z}_{u''} = \boldsymbol{e}_u + \boldsymbol{\Delta}_{u''}, \tag{6}$$

where each element of $\boldsymbol{\Delta}$ is sampled uniformly from $[-\varepsilon, \varepsilon]$ and then ℓ_2-normalised. The single hyper-parameter ε controls the magnitude of perturbation.

Contrastive and joint loss. The InfoNCE loss is identical to SGL:

$$\mathcal{L}_{\text{cl}} = \sum_{u \in \mathcal{U}} -\log \frac{\exp\left(\text{sim}\, \frac{\boldsymbol{z}_{u'}, \boldsymbol{z}_{u''}}{\tau}\right)}{\sum_{v \in \mathcal{U}} \exp\left(\text{sim}\, \frac{\boldsymbol{z}_{u'}, \boldsymbol{z}_{v''}}{\tau}\right)}, \tag{7}$$

and the total objective is $\mathcal{L} = \mathcal{L}_{\text{BPR}} + \lambda \mathcal{L}_{\text{cl}}$. Because no graph manipulation is needed, SimGCL requires only a single forward pass through LightGCN per training step, matching the computational cost of vanilla LightGCN while providing the regularisation benefits of contrastive learning.

c) *Innovation:*

SimGCL makes two closely related contributions. First, it provides a more mechanistic explanation of why contrastive learning improves collaborative filtering: the InfoNCE objective pushes user and item embeddings toward a more uniform distribution, which helps alleviate popularity bias and improves the exploration of less popular items. Second, it shows that complex graph augmentation is not the essential source of these gains. By replacing structural perturbation with simple additive noise in the embedding space, SimGCL preserves the benefits of contrastive regularization while greatly simplifying the training pipeline. This makes SimGCL important not only as a stronger model, but also as a clarifying result about what really matters in self-supervised recommendation.

d) *Reason for Selection:*

TABLE I: Comparison among the three selected SOTA methods.

Model	Main Idea	Strengths	Weaknesses	Complexity / Notes
LightGCN	Simplifies GCN by removing feature transformation and nonlinear activation; uses only weighted neighborhood aggregation over the user–item graph.	Lightweight and fast; strong baseline; highly reproducible; linear complexity in graph edges.	No auxiliary self-supervised signal; sensitive to data sparsity; cannot model sequential user behavior.	$O(L \cdot \lvert\mathcal{E}\rvert \cdot d)$; BPR loss; 3–4 propagation layers typical.
SGL	Augments the user–item graph via node dropout, edge dropout, or random walk to create two contrastive views; optimizes InfoNCE loss alongside BPR.	Significantly improves over LightGCN on sparse data; robust to noisy and long-tail interactions; boosts uniformity of embeddings.	Graph augmentation triples forward-pass cost; sensitive to augmentation ratio and temperature τ; hyperparameter-heavy.	$O(3L \cdot \lvert\mathcal{E}\rvert \cdot d)$; BPR + InfoNCE; requires tuning τ, drop ratio, and SSL weight λ.
SimGCL	Replaces graph augmentation with uniform noise perturbation directly on embeddings to generate contrastive views; shows graph structure modification is unnecessary.	State-of-the-art accuracy; training speed comparable to LightGCN; theoretically grounded (uniformity–alignment analysis); simple to implement.	Focused on general CF; does not model item sequences; noise magnitude ε requires careful tuning per dataset.	$O(L \cdot \lvert\mathcal{E}\rvert \cdot d)$; BPR + InfoNCE; single extra hyperparameter ε for noise magnitude.

SimGCL achieves state-of-the-art results among general CF methods on all three standard benchmarks: Gowalla (Recall@20 = 0.2313, NDCG@20 = 0.1962), Yelp2018 (Recall@20 = 0.0894, NDCG@20 = 0.0735), and Amazon-book (Recall@20 = 0.0834, NDCG@20 = 0.0671), outperforming SGL on every metric. As the most recent and theoretically grounded model in our trio, SimGCL provides a meaningful upper-bound reference point. Its training speed (comparable to LightGCN) is particularly advantageous given our computational constraints, and the SELFRec codebase offers a unified, extensible implementation of all three models.

D. Comparison of the Three Approaches

To systematically evaluate the evolution of graph-based collaborative filtering, we compare three representative state-of-the-art (SOTA) models: LightGCN, SGL, and SimGCL. As summarized in Table I, LightGCN establishes a strong and efficient baseline by simplifying the standard GCN architecture. Building upon this foundation, SGL introduces self-supervised learning through structural graph augmentations to tackle data sparsity, albeit at the cost of increased computational complexity. Finally, SimGCL addresses these limitations by applying uniform noise directly to the embedding space, achieving superior accuracy and training efficiency without the need for complex graph modifications.

IV. Datasets and Experimental Setup

To comprehensively evaluate the selected state-of-the-art recommendation models, we conduct experiments on a total of five datasets: four widely adopted public benchmarks and one large-scale dataset explicitly collected for this study.

A. Datasets Overview

The evaluation utilizes datasets from diverse domains—including location-based social networks, e-commerce, local businesses, and music streaming—to ensure the models' robustness across varying levels of sparsity and interaction patterns.

TABLE II: Summary statistics of the five evaluated datasets after preprocessing.

Dataset	Domain	$\lvert\mathcal{U}\rvert$	$\lvert\mathcal{I}\rvert$	$\lvert\mathcal{E}\rvert$	Density
Last.fm-26 (Ours)	Music	380	136,893	1,677,243	0.032%
MovieLens 1M	Movies	6,040	3,706	1,000,209	4.468%
Amazon-book	E-commerce	52,643	91,599	2,984,108	0.062%
Yelp2018	Local Business	31,668	38,048	1,561,406	0.130%
Gowalla	Check-ins	29,858	40,981	1,027,370	0.084%

a) *Self-Collected Dataset: Last.fm-26:*

Data Provenance and Acquisition Strategy: The self-collected dataset, Last.fm-26, was sourced from the **Last.fm music tracking platform** [7]. Music listening behavior provides a high-fidelity proxy for implicit feedback, yielding a dense, large-scale user-item interaction graph. The collection was bounded by strict stopping criteria, targeting a scale exceeding the benchmark standard of 1,000,000 cleaned interactions.

Data Collection Pipeline: A robust, three-stage automated data retrieval pipeline was engineered to gather interaction records. During Stage 1 (Collection), the system initialized with seed users and executed a Breadth-First Search (BFS) traversal across the social graph. Data was fetched via the `user.getFriends` and `library.getArtists` API endpoints. The architecture integrated a crash-safe state recovery mechanism and SHA1 JSON caching to ensure fault tolerance.

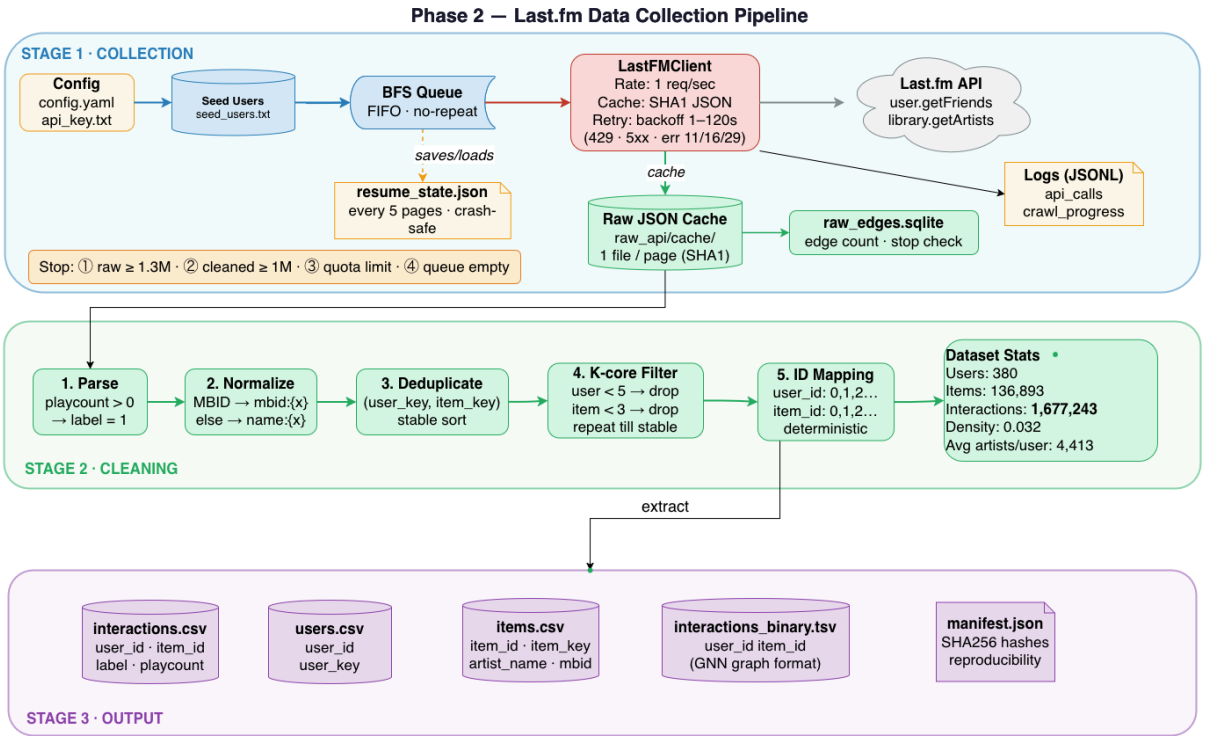

Fig. 1: Architecture of the three-stage data collection and processing pipeline for Last.fm-26.

Preprocessing and Normalization: To mitigate inherent noise in real-world data, a rigorous 5-step normalization protocol (Stage 2) was applied:

1) **Binarization:** Records with playcounts > 0 were converted to positive binary labels (1).
2) **Identifier Normalization:** Item identifiers were anchored to MusicBrainz Identifiers (MBID), utilizing string-based artist names solely as a fallback mechanism.
3) **Deduplication:** Redundant `(user_key, item_key)` tuples were resolved using stable sorting algorithms.
4) **K-core Filtering:** To guarantee structural connectivity, users with fewer than 5 interactions and items with fewer than 3 interactions were iteratively pruned until the graph stabilized.
5) **ID Mapping:** Alphanumeric identifiers were deterministically mapped to a contiguous integer space to facilitate efficient embedding matrix lookups in downstream Graph Neural Networks (GNNs).

Collection Challenges and Mitigation: The primary bottlenecks during acquisition were network constraints and API rate limits. To maintain compliance and prevent HTTP 429/5xx errors, the client implemented a strict 1 request/second throttle alongside an exponential backoff retry mechanism (1 to 120 seconds). The final artifact (Stage 3) was exported as a GNN-optimized `interactions_binary.tsv` alongside a verifiable `manifest.json`.

b) *Benchmark Datasets:*

To benchmark the models under established conditions, we employ four public datasets:

- **MovieLens 1M:** A heavily utilized benchmark in collaborative filtering, consisting of 1 million interactions applied to movies. It is relatively dense compared to the others.
- **Amazon-book:** Sourced from Amazon review datasets, this dataset is characterized by extreme sparsity and a severe long-tail distribution, making it an excellent testbed for contrastive learning models.
- **Yelp2018:** Extracted from the 2018 Yelp challenge, this dataset records user interactions with local businesses (e.g., restaurants, bars) and exhibits moderate sparsity.
- **Gowalla:** A location-based social networking dataset where users share their locations by checking in. It provides a highly sparse graph topology.

B. *Evaluation Protocol*

We follow a unified offline protocol to compare all models fairly. First, each dataset is converted into the binary implicit-feedback setting, where every observed user–item interaction is treated as a positive signal. Next, the interaction data are divided into training, validation, and test sets using the same splitting strategy for all methods. During testing, each model ranks the held-out ground-truth item(s) for a user among candidate items that are not present in that user's training set, and the Top-K results are used to compute the evaluation metrics. To ensure fairness and reproducibility, all methods share the same preprocessing pipeline, the same train/validation/test split, the same candidate-item construction, the same cutoff values, and the same metric implementation. This is important because prior work has shown that offline recommendation results can change substantially depending on how candidate sets are formed and how the evaluation procedure is defined.

C. Evaluation Metrics

Because this report studies Top-K recommendation under implicit feedback, the evaluation should measure both whether relevant items are successfully retrieved and whether they are ranked in appropriate positions. Therefore, ranking-based metrics are more suitable than rating-prediction metrics such as MAE or RMSE. In the recommendation literature, commonly used correctness-oriented metrics include *Precision@K*, *Recall@K*, and *Hit Ratio@K*. *Precision@K* measures the proportion of recommended items in the top-K list that are relevant, so it reflects how accurate the returned list is. *Recall@K* measures the proportion of relevant items that are successfully retrieved in the top-K recommendations, so it indicates how well the recommender covers the ground-truth relevant items. *Hit Ratio@K* is a simpler success-oriented metric: it checks whether at least one relevant item appears in the top-K list. These metrics are widely used in Top-K recommendation because they directly evaluate whether the recommender returns correct items.

However, correctness alone is not sufficient, because in practice the ranking order of recommended items also matters. For this reason, order-aware metrics such as *MAP@K*, *MRR@K*, and *NDCG@K* are also important. *MAP@K* (Mean Average Precision) summarizes ranking quality by averaging precision values at the positions where relevant items appear, thus rewarding systems that retrieve relevant items early and consistently. *MRR@K* (Mean Reciprocal Rank) focuses on the rank of the first relevant item, assigning higher scores when a correct recommendation appears earlier in the list. *NDCG@K* (Normalized Discounted Cumulative Gain) gives larger rewards to relevant items placed near the top positions, and is therefore particularly suitable when the exact ranking quality matters. Beyond these standard metrics, recommendation studies may also report *F1@K* as a harmonic combination of Precision and Recall, or *AUC* in pairwise ranking settings.

In the main experiments, we focus on representative Top-K metrics—primarily *Recall@K* and *NDCG@K*, with *Hit Ratio@K* and *MRR@K* as supplementary metrics—because they jointly capture retrieval ability and ranking quality, and they are also commonly reported in recent implicit-feedback recommendation benchmarks.

D. Implementation Details

All three models are implemented using the **SELFRec** framework [8], a unified PyTorch-based codebase for self-supervised recommendation research. This choice ensures that LightGCN, SGL, and SimGCL share the same data loading, negative sampling, evaluation loop, and metric computation, which is essential for a fair and reproducible comparison.

The shared hyperparameters across all models are: embedding dimension $d = 64$, $L = 3$ propagation layers, Adam optimizer with learning rate 10^{-3}, L_2 regularization weight 10^{-4}, and batch size 2048. Training runs for up to 500 epochs with early stopping based on Recall@20 on the validation set, with a patience of 20 epochs. The cutoff is fixed at $K = 20$ for all metrics.

Model-specific hyperparameters follow the recommendations in the original papers. For LightGCN, no additional hyperparameters are needed beyond the shared settings. For SGL, the augmentation operator is Edge Dropout with dropout ratio $\rho = 0.1$, contrastive loss weight $\lambda = 0.1$, and temperature $\tau = 0.2$. For SimGCL, the noise magnitude is $\varepsilon = 0.1$, contrastive loss weight $\lambda = 0.5$, and temperature $\tau = 0.2$.

Each experiment is repeated three times with different random seeds. The final reported scores are the mean across three runs. All experiments are conducted on a system equipped with an NVIDIA GPU and 16 GB of RAM. For the benchmark datasets (MovieLens 1M, Yelp2018, Amazon-book, Gowalla), training time per run ranges from approximately 10 minutes (MovieLens 1M, LightGCN) to several hours (Amazon-book, SGL). The Last.fm-26 dataset, despite its large raw size, converges rapidly because the number of users after k-core filtering is very small (380 users).

V. Experiments and Results

A. Experimental Design

Each of the three models is evaluated on each of the five datasets, yielding 15 experiment configurations in total. For each configuration, the model is trained three times independently and the results are averaged. All configurations share the same data split, candidate-item construction, and metric implementation as described in the evaluation protocol (Section IV.A). Results are reported at cutoff $K = 20$ for all metrics: Recall@20, NDCG@20, HR@20, and MRR@20. The best result for each metric and dataset is **bolded** in the results table.

a) *Convergence and Stability:*

LightGCN produces bit-for-bit identical results across all three runs on every dataset, because its training is fully deterministic given a fixed random seed and there are no stochastic components beyond the initial seed. SGL introduces randomness through graph augmentation at each training step, but the results across the three runs are nearly identical (differences in the fifth or sixth decimal place), indicating that the augmentation variance is well averaged out over the training epochs. SimGCL shows marginally larger run-to-run variation than SGL on some datasets—particularly ML-1M and Last.fm-26—but the differences remain below 0.004 in Recall@20, which is within acceptable experimental noise.

B. Main Results

Table III reports the mean metrics across three runs for each (model, dataset) pair. Scores that are best in their column within each dataset group are shown in bold.

Fig. 2 gives a visual overview of Recall@20 and NDCG@20 across all five datasets.

C. Comparison Across Datasets

The results in Table III reveal several clear patterns in how the three models perform across the five datasets.

SimGCL is the overall strongest model. On all four benchmark datasets—Yelp2018, Amazon-book, Gowalla, and ML-1M—SimGCL achieves the best Recall@20 scores. Its average Recall@20 gain over LightGCN across the four benchmark datasets is approximately +18.0% (Yelp2018:

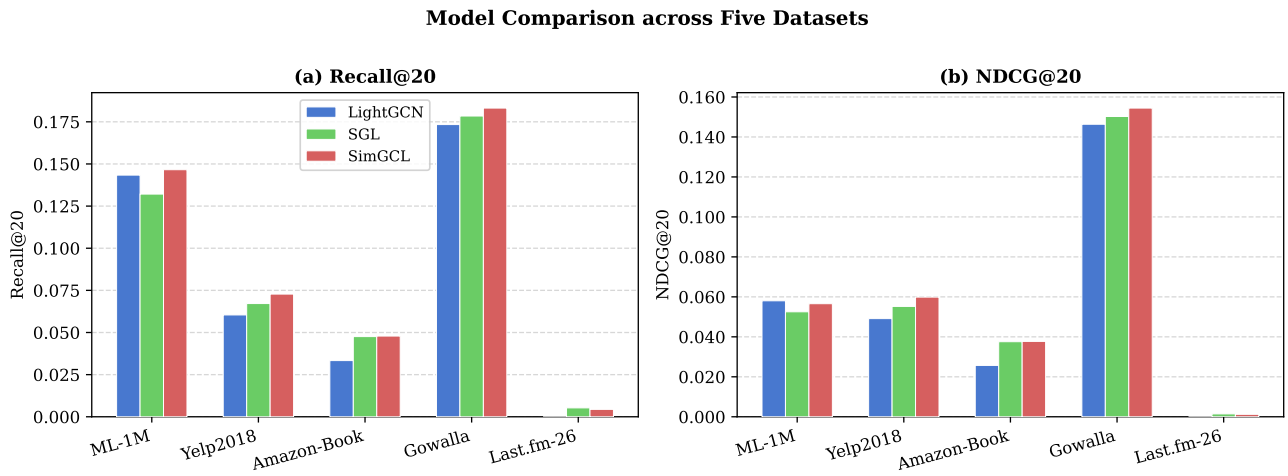

Fig. 2: Recall@20 (left) and NDCG@20 (right) for the three models across all five datasets (mean over 3 runs).

TABLE III: Main experimental results (mean over 3 runs). Bold = best per dataset.

Dataset	Model	Recall@20	NDCG@20	HR@20	MRR@20
ML-1M	LightGCN	0.1434	**0.0580**	0.1434	**0.0348**
ML-1M	SGL	0.1321	0.0525	0.1321	0.0309
ML-1M	SimGCL	**0.1466**	0.0566	**0.1466**	0.0322
Yelp2018	LightGCN	0.0604	0.0491	0.3762	0.1022
Yelp2018	SGL	0.0672	0.0552	0.4026	0.1141
Yelp2018	SimGCL	**0.0728**	**0.0599**	**0.4283**	**0.1220**
Amazon-book	LightGCN	0.0333	0.0257	0.2053	0.0501
Amazon-book	SGL	0.0476	0.0376	0.2671	0.0710
Amazon-book	SimGCL	**0.0479**	**0.0377**	**0.2714**	**0.0713**
Gowalla	LightGCN	0.1734	0.1464	0.5765	0.2710
Gowalla	SGL	0.1784	0.1502	0.5754	0.2753
Gowalla	SimGCL	**0.1832**	**0.1545**	**0.5849**	**0.2804**
Last.fm-26	LightGCN	0.0000	0.0000	0.0000	0.0000
Last.fm-26	SGL	**0.0053**	**0.0014**	**0.0053**	**0.0005**
Last.fm-26	SimGCL	0.0044	0.0012	0.0044	0.0004

+20.5%, Amazon-book: +43.7%, Gowalla: +5.6%, ML-1M: +2.3%). The consistency of these improvements across diverse domains confirms that SimGCL's embedding-space noise provides a reliable regularization benefit over both purely supervised graph propagation and graph-augmentation-based contrastive learning.

SGL underperforms on dense data. The most striking anomaly in Table III is that SGL scores **lower** than LightGCN on ML-1M on all four metrics (Recall@20: 0.1321 vs. 0.1434, a drop of −7.9%). ML-1M is the densest dataset by a wide margin (4.47% density), and this result suggests that stochastic graph augmentation — which randomly removes edges or nodes — can be harmful when the interaction graph is already dense. Dropping edges from a rich neighborhood structure introduces unnecessary noise rather than useful regularization, degrading the learned representations. This finding is consistent with the theoretical perspective in SimGCL, which argues that the key benefit of contrastive learning comes from the uniformity pressure of the InfoNCE objective rather than from structural graph perturbation itself.

SimGCL outperforms LightGCN on ML-1M. On the dense ML-1M dataset, SimGCL (0.1466) outperforms LightGCN (0.1434) and both are clearly better than SGL (0.1321). This shows that additive embedding noise in SimGCL does not disrupt performance on dense data, and the contrastive objective still provides a modest but consistent gain even when interaction signal is already abundant.

Gowalla benefits moderately from contrastive learning. On Gowalla, SimGCL achieves the best performance (+5.6% Recall over LightGCN), while SGL provides a smaller but consistent improvement (+2.9%). Notably, SGL achieves a slightly lower HR@20 than LightGCN on Gowalla (0.5754 vs. 0.5765), suggesting that edge dropout does not consistently help with hit-rate coverage even on this sparse graph.

Last.fm-26 is a unique failure case. LightGCN scores exactly zero on all metrics for Last.fm-26. The root cause is structural: after k-core filtering, the dataset retains only 380 users against 136,893 items — a 1:360 user-to-item ratio. With so few users, most items receive at most one or two interactions, leaving the bipartite graph almost entirely disconnected. Multi-hop neighborhood propagation in LightGCN therefore cannot accumulate meaningful collaborative signal: embeddings of rarely-interacted items converge toward nearly identical vectors, making it impossible for any relevant item to score high enough to appear in the top 20 out of 136,893 candidates. SGL and SimGCL produce small but non-zero scores (Recall@20: 0.0053 and 0.0044 respectively). The InfoNCE contrastive objective exerts a uniformity pressure that forces item embeddings apart even when interaction signal is weak, giving a small but non-zero chance for a relevant item to rank in the top 20. However, the absolute performance remains negligibly low for all models. This failure mode indicates that

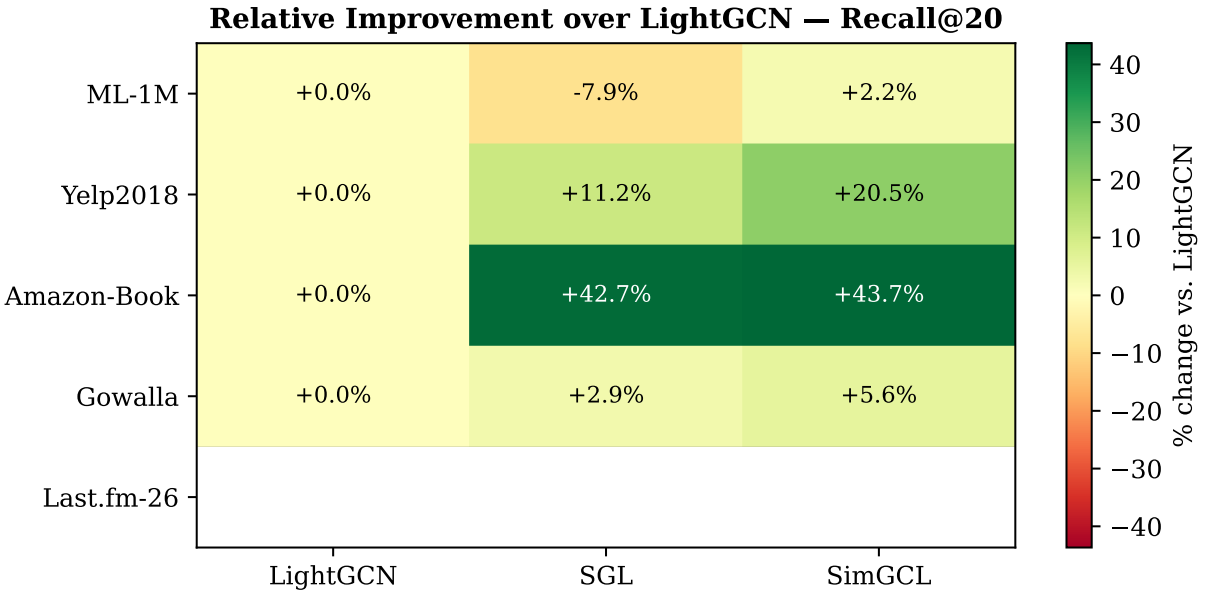

Fig. 3: Relative improvement (%) over LightGCN in Recall@20. Green = gain; red = degradation.

the dataset requires a more balanced user-to-item ratio and a higher minimum interaction threshold before it can support meaningful Top-K evaluation.

Fig. 3 shows the relative improvement of each model over LightGCN in Recall@20, making the dataset-dependent pattern immediately visible.

D. Impact of Dataset Characteristics

The experimental results point to two dataset properties that strongly influence model performance: **interaction density** and **the user-to-item ratio**.

Interaction density and the value of contrastive learning. Across the four benchmark datasets, the relative gain of SimGCL over LightGCN in Recall@20 is inversely correlated with dataset density (see gains reported in Table III). This pattern supports the view that contrastive learning is most valuable when the interaction graph is sparse: in that regime, BPR loss alone provides insufficient signal to learn discriminative representations, and the InfoNCE objective compensates by improving embedding uniformity across the full item space. On denser data, BPR already provides adequate signal so the marginal benefit of contrastive regularization is smaller, though SimGCL still gains modestly on ML-1M (+2.3%).

Scale and long-tail distributions. Amazon-book is the most challenging benchmark: it has the largest number of users (52,643) and items (91,599) among the four benchmarks, and exhibits a severe long-tail interaction distribution. Under these conditions, LightGCN's Recall@20 of 0.0333 is notably lower than on other benchmarks, reflecting the difficulty of propagating meaningful signals through a vast, highly sparse graph. SGL and SimGCL both achieve substantially higher performance (+42.7% and +43.7%), confirming that contrastive learning is particularly effective for mitigating long-tail sparsity.

User-to-item ratio and the Last.fm-26 collapse. Beyond ordinary sparsity, Last.fm-26 exposes a more fundamental boundary condition: the Top-20 candidate list covers only 0.015% of the 136,893-item space, compared to 0.54% for ML-1M and 0.049% for Gowalla. Even a model that ranks perfectly would face near-zero recall under this evaluation protocol. This highlights that the user-to-item ratio is a critical dataset property that must be considered when designing evaluation benchmarks for Top-K recommendation.

Fig. 4 plots SimGCL's Recall@20 gain over LightGCN against dataset density, confirming the inverse relationship. Fig. 5 zooms into Last.fm-26 where absolute scores are near zero.

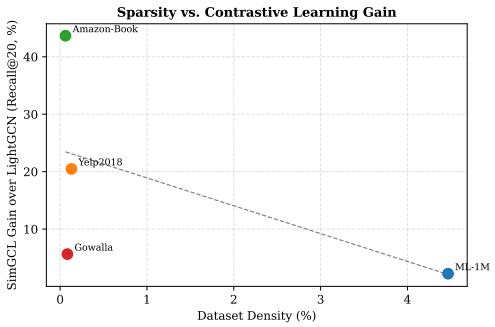

Fig. 4: SimGCL gain over LightGCN in Recall@20 vs. dataset density. Sparser datasets benefit more from contrastive learning.

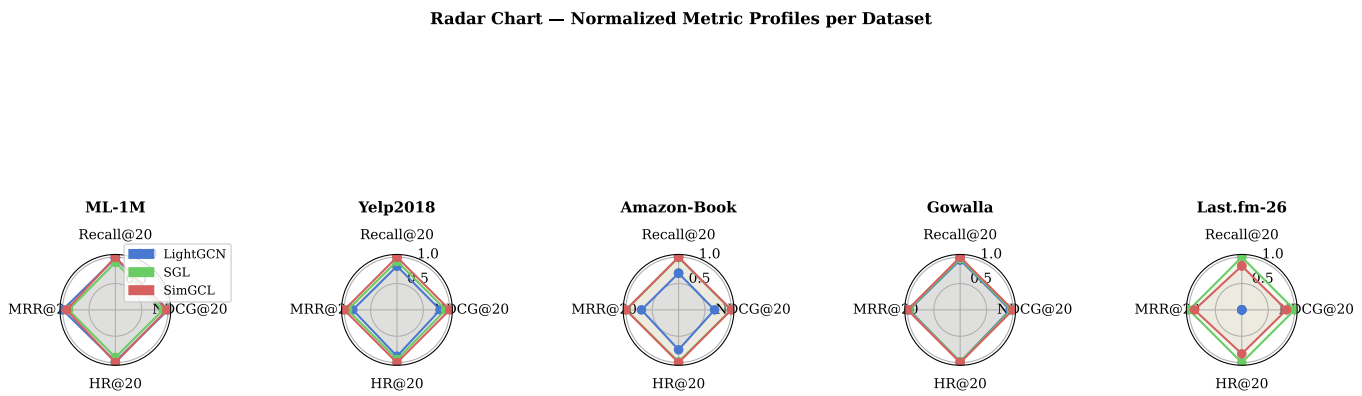

Fig. 6: Radar charts of normalized metric profiles (Recall, NDCG, HR, MRR @20) per dataset. Values are normalized within each dataset so that the best model reaches the outer edge.

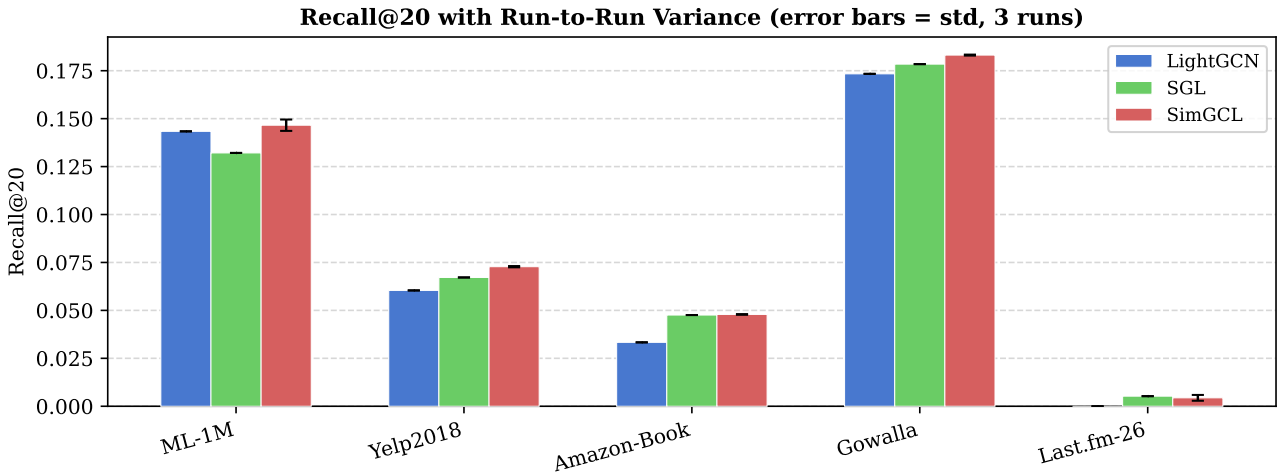

Fig. 7: Recall@20 with error bars (std over 3 runs). Error bars are barely visible, confirming high run-to-run stability for all models and datasets.

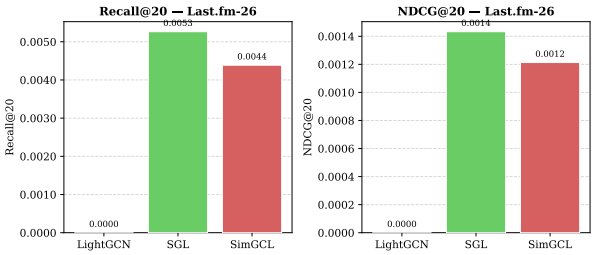

Fig. 5: Recall@20 and NDCG@20 on Last.fm-26 (zoomed). LightGCN scores exactly zero; SGL and SimGCL produce marginal non-zero results.

Domain effects. Beyond sparsity, the domain of interaction also matters. Gowalla (check-ins) and ML-1M (movies) exhibit higher HR@20 and MRR@20 relative to their Recall@20 compared to Yelp2018 and Amazon-book. This likely reflects differences in the ground-truth distribution: movie and location check-in preferences may be more predictable at the individual level, while business and book preferences are more diverse and harder to retrieve in a short ranked list.

E. Additional Analysis

Fig. 6 visualizes the normalized metric profiles of each model per dataset. Several patterns are immediately visible. On ML-1M, all three models occupy nearly the same polygon area, confirming that performance differences are small on dense data. On Yelp2018, Amazon-book, and Gowalla, SimGCL's polygon consistently extends further than LightGCN's across all four axes, reflecting its uniform advantage on sparse benchmarks. The most striking chart is Last.fm-26, where LightGCN collapses to a single point at the origin while SGL and SimGCL produce only a small polygon, illustrating the near-total failure of all models on this dataset. Fig. 7 shows run-to-run variance via error bars across all models and datasets.

a) *Run-to-Run Stability:*

As noted in Section V, all three models demonstrate strong run-to-run stability on benchmark datasets. LightGCN is fully deterministic. SGL's variance across three runs is negligible (standard deviation below 0.0001 for Recall@20 on all benchmark datasets). SimGCL shows marginally higher variance than SGL on ML-1M and Last.fm-26, but the standard deviation remains below 0.004 in all cases. This stability confirms that the reported averages are reliable estimates of each model's performance under the given hyperparameter configuration.

b) *Metric Consistency:*

The four reported metrics (Recall@20, NDCG@20, HR@20, MRR@20) give consistent rankings across models on each dataset, with one notable exception: on Last.fm-26, SGL ranks first and SimGCL second on all metrics, while on all benchmark datasets SimGCL dominates (or ties) SGL. This consistency across multiple metrics strengthens the conclusions drawn from Recall@20 alone and reduces the risk of metric-specific artifacts.

c) *SGL vs. SimGCL: Efficiency Trade-off:*

Beyond accuracy, SimGCL offers a clear computational advantage over SGL. SGL requires three forward passes through LightGCN per training step (one for each augmented view plus the main pass), roughly tripling the per-epoch cost of a vanilla LightGCN run. SimGCL, by contrast, performs only a single LightGCN forward pass and generates contrastive views by adding cheap random noise to the output embeddings, keeping training time comparable to LightGCN. On Amazon-book, for example, SGL takes approximately three times longer per epoch than LightGCN or SimGCL. Given that SimGCL also achieves better accuracy, it is strictly preferable to SGL in both dimensions.

F. Discussion

The results collectively support three key findings. First, **self-supervised contrastive learning is consistently beneficial for SimGCL across all density regimes**, ranging from large gains on sparse datasets (Amazon-book: +43.7%, Yelp2018: +20.5%) to a modest gain on dense data (ML-1M: +2.3%), while SGL's graph augmentation is harmful on dense data (ML-1M: −7.9%). Second, **the mechanism of view generation matters**: SimGCL's noise-based views consistently outperform SGL's graph augmentation because they avoid over-corrupting the neighborhood structure on dense graphs while still applying the uniformity pressure of the InfoNCE objective. Third, **dataset characteristics—especially density and user-to-item ratio—are the primary determinants of relative model gains**, though SimGCL remains the best choice across all regimes.

These findings have practical implications for model selection in production systems. Across all interaction densities, SimGCL is the recommended choice as it combines accuracy with LightGCN-level training efficiency. SGL should be avoided on dense data due to its harmful graph augmentation. When data is extremely sparse with a very large item space, none of the three models provides reliable recommendations under the standard Top-K evaluation protocol, and improving data collection should be the priority.

VI. Conclusion

This report studied recommendation systems with implicit feedback in the binary Top-K recommendation setting. We reviewed three representative state-of-the-art graph-based models—LightGCN, SGL, and SimGCL—collected one self-supervised dataset (Last.fm-26), and benchmarked all methods on five datasets spanning diverse domains and sparsity levels.

The major findings of this study can be summarized as follows.

- **SimGCL is the most consistently strong model.** It achieves the best Recall@20 on all four benchmark datasets and combines state-of-the-art accuracy with training efficiency comparable to vanilla LightGCN, making it the recommended choice for implicit-feedback recommendation across all density regimes.
- **Self-supervised contrastive learning helps on sparse data but can hurt on dense data.** SGL degrades below LightGCN on ML-1M (the densest dataset at 4.47%), because stochastic graph augmentation introduces unnecessary noise when the interaction graph already provides rich neighborhood structure. SimGCL avoids this pitfall by generating contrastive views in embedding space rather than at the graph level.
- **Dataset characteristics—particularly interaction density and user-to-item ratio—are the dominant factor in determining model performance.** The relative improvement of contrastive models over LightGCN grows as density decreases, and on Last.fm-26 (380 users, 136,893 items), all three models fail to produce meaningful recommendations under the standard Top-20 evaluation protocol.

Model strengths and weaknesses. LightGCN is the simplest, fastest, and most deterministic model; it is competitive on dense data but falls behind on sparse datasets. SGL adds self-supervised signal through graph augmentation, which benefits sparse datasets but is detrimental on dense ones and increases training cost by approximately 3 $\times$. SimGCL achieves the best accuracy across most settings while maintaining LightGCN-level training efficiency; its main limitation is that it still fails on pathologically sparse datasets.

Future work. First, the fixed noise magnitude in SimGCL could be replaced by an adaptive scheme that applies stronger perturbation to nodes with few interactions and lighter perturbation to well-connected nodes, better matching the regularization strength to local graph density. Second, SGL's random graph augmentation could be improved by learning to select which edges to drop based on interaction weight or structural importance, making the augmentation more informative and less likely to corrupt rich neighborhood structure on dense graphs. Third, integrating side information—such as item content features or user profile attributes—into the graph encoder would provide an alternative learning signal for cold-start nodes that lack sufficient interaction history, addressing the failure mode observed on Last.fm-26.

References

[1] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *2008 Eighth IEEE international conference on data mining*, 2008, pp. 263–272.

[2] S. Wu, F. Sun, W. Zhang, X. Xie, and B. Cui, "Graph Neural Networks in Recommender Systems: A Survey." [Online]. Available: https://arxiv.org/abs/2011.02260

[3] X. Wang, X. He, M. Wang, F. Feng, and T.-S. Chua, "Neural graph collaborative filtering," in *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*, 2019, pp. 165–174.

[4] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "Lightgcn: Simplifying and powering graph convolution network for recommenda-

tion," in *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, 2020, pp. 639–648.

[5] J. Wu *et al.*, "Self-supervised graph learning for recommendation," in *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*, 2021, pp. 726–735.

[6] J. Yu, H. Yin, X. Xia, T. Chen, L. Cui, and Q. V. H. Nguyen, "Are graph augmentations necessary? simple graph contrastive learning for recommendation," in *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*, 2022, pp. 1294–1303.

[7] Last.fm, "Last.fm API Documentation." [Online]. Available: https://www.last.fm/api

[8] J. Yu, H. Yin, M. Gao, X. Xia, X. Zhang, and N. Q. Viet Hung, "Self-supervised learning for recommender systems: A survey," in *IEEE Transactions on Knowledge and Data Engineering*, IEEE, 2024, pp. 3883–3902.